

ASSIGNMENT

Program- IoT and Drone Building

Name- Tanvi Kohli

Roll Number- IoT026

Q1) You are building a smart irrigation system that must monitor soil moisture, control a water pump, and send data to a mobile application every 5 seconds. You have both an Arduino UNO and an ESP32. Which controller would you choose and why? Would your answer change if internet connectivity was not required?

Ans- For a smart irrigation system that monitors soil moisture, controls a pump, and sends data every 5 seconds, I would choose the **ESP32**. This is mainly because it has **built-in Wi-Fi**, making it ideal for IoT applications. It can handle sensor readings, pump control, and data transmission efficiently due to its faster processor and multitasking capability. It also avoids the need for extra modules, reducing complexity.

In contrast, the **Arduino UNO** lacks native internet connectivity and would require additional hardware, making the system more complex.

However, if internet connectivity is not required, I would choose the **Arduino UNO**, since it is simple, reliable, and sufficient for basic sensing and control tasks.

In conclusion, **ESP32 is better for IoT-based systems**, while **Arduino UNO is suitable for standalone applications**.

Q2) A smart factory contains 500 sensors distributed across a 2 km area. Compare Wi-Fi, Bluetooth, LoRa, and MQTT-based communication. Which protocol or combination of protocols would you choose and why?

Ans- In a smart factory with 500 sensors over a 2 km area, the communication protocol must balance range, power, and scalability.

Wi-Fi offers high data rates but has limited range and high power consumption, requiring many access points, which increases cost and complexity.

Bluetooth (BLE) is low power but has very short range, making it unsuitable for large industrial areas.

LoRa, on the other hand, provides long-range communication (several kilometers) with low power usage and is ideal for sending small sensor data efficiently.

MQTT is an application-layer protocol used for reliable data transfer over networks, typically on top of Wi-Fi or Ethernet.

Based on this, I would choose a **combination of LoRa and MQTT**. Sensors would use LoRa to send data to a central gateway, and the gateway would forward it to the cloud using MQTT.

This approach ensures **long-range, low-power communication at the sensor level and efficient data handling at the system level**, making it scalable and suitable for large factory environments.

Q3) A temperature sensor suddenly starts showing impossible values (e.g., -120°C or 300°C). How you would determine whether the problem lies in the sensor, wiring, power supply, ADC, or software.

Ans-If a temperature sensor starts showing impossible values like -120°C or 300°C , I would debug it step by step.

First, I would check the **sensor itself** by replacing it or testing it in another setup to see if it is faulty.

Next, I would inspect the **wiring and connections**, ensuring all pins (VCC, GND, signal) are secure and correctly connected.

Then, I would verify the **power supply** using a multimeter to confirm stable and correct voltage levels.

If it is an analog sensor, I would check the **ADC configuration**, including resolution and reference voltage, and observe raw values.

Finally, I would review the **software**, checking for errors in conversion formulas, calibration, or data handling.

In this way, I can systematically identify whether the issue is due to the sensor, hardware, or software.

Q4) An ESP32-based weather station must operate on battery power for six months without maintenance. What hardware and software modifications would you make to maximize battery life?

Ans- For an ESP32-based weather station that must run on battery for six months, the main goal is to **minimize power consumption**.

From a **hardware perspective**, I would use **low-power sensors** and avoid always-on components like LEDs or displays. An efficient **power supply (buck/boost converter)** would be preferred over linear regulators. I would also use a **high-capacity battery** and optionally a small solar panel. Additionally, sensors can be powered only when needed using transistors or MOSFETs.

From a **software perspective**, I would heavily use the ESP32's **deep sleep mode**, waking it only at intervals (e.g., every 5–10 minutes) to take readings and transmit data. Since Wi-Fi consumes high power, I would **reduce transmission frequency** or

send data in batches. The code would also be optimized to keep the processor active for the shortest time possible.

Q5) An automatic braking system in a vehicle requires communication with almost zero delay. Which communication method would you choose and why?

Ans- For an automatic braking system, the key requirement is **almost zero delay with high reliability**, since even small delays can be dangerous.

I would choose the **CAN (Controller Area Network) bus** for communication. It is designed for **real-time automotive systems** and provides **low latency and deterministic communication**, ensuring data is delivered within a predictable time.

Another advantage is its **priority-based message system**, where critical signals like braking are transmitted first without delay. Additionally, CAN is **highly reliable and resistant to noise**, with built-in error detection, making it suitable for harsh vehicle environments.

Therefore, CAN is an ideal choice for ensuring **fast, reliable, and safe communication** in braking systems.

Q6) You need to measure the average temperature of a classroom using only one sensor. Where would you place the sensor and why? Explain how poor placement could affect the accuracy of your measurements. Assume the classroom to be a square-base cuboid of side length “a” and height “b”.

Ans- To measure the average temperature of a classroom using one sensor, I would place it at the **geometric center of the room**, i.e., at approximately $(a/2, a/2, b/2)$. Practically, this means positioning it **centrally and at mid-height**, away from walls, floor, and ceiling.

This location is ideal because it minimizes the influence of external factors like walls, sunlight, airflow, and heat sources, giving the best approximation of the average temperature.

Poor placement can reduce accuracy:

- Near windows → higher readings (sunlight)
- Near doors/vents → fluctuating or lower readings
- Near ceiling → higher readings (hot air rises)
- Near floor → lower readings (cool air settles)
- Near equipment/people → localized heating

Thus, improper placement leads to **biased measurements** that do not represent the true average temperature of the room.

Q7) Under what situations could an analog sensor perform better than a digital one?

Ans- Although digital sensors are generally easier to use, analog sensors can perform better in certain situations.

Firstly, analog sensors are useful for **continuous and high-speed measurements**, as they provide instant voltage changes without communication delays. This makes them suitable for real-time systems.

Secondly, they can offer **higher resolution**, since their output is continuous and not limited to discrete digital steps (especially when used with a high-resolution ADC).

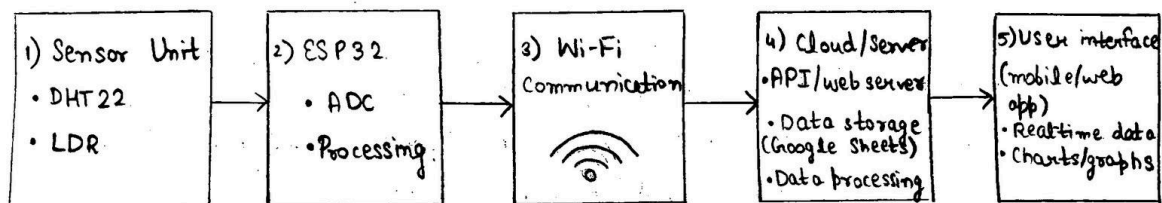
They are also beneficial in **low-latency or simple systems**, as they do not require protocols like I2C or SPI, reducing overhead.

Additionally, in some **noisy environments**, analog sensors can still provide usable signals with proper filtering, whereas digital communication may fail completely.

Q8) Draw the complete architecture/block diagram of your IoT project, beginning from the sensor and ending at the user interface. Identify every possible point where a failure could occur and suggest one solution for each.

Ans-

Ans. Block Diagram. (Architecture).



2. Explanation

- **Sensors:** Measure temperature, humidity, and light
- **ESP32:** Reads data, processes it, and controls devices
- **Wi-Fi:** Sends data to the cloud
- **Cloud/Server:** Stores and manages data
- **User Interface:** Displays real-time data to the user

3. Failure Points & Solutions

- **Sensor failure:** Use backup or error checking
- **Wiring issues:** Ensure proper connections
- **Power supply:** Use stable regulated supply

- **Noise/ADC errors:** Apply filtering
- **ESP32 failure:** Use watchdog timer
- **Wi-Fi issues:** Auto-reconnect + temporary storage
- **Server failure:** Retry or use backup storage
- **Data errors:** Use reliable protocols
- **UI issues:** Add validation and refresh

Conclusion

The system works as a **sensor** → **processing** → **communication** → **user interface pipeline**, and reliability can be improved using **error handling and backup mechanisms**.

Q9) If an attacker gains access to your IoT device through the Wi-Fi network, what kinds of attacks could they perform? Suggest at least five methods to improve the security of the system (with applicable solutions, do not cite far-fetched ideas)

Ans- **Possible Attacks**

- **Unauthorized control:** Attacker operates devices like pumps or relays
- **Data theft:** Intercepts sensor data, credentials, or API keys
- **Data manipulation:** Alters readings, causing wrong decisions
- **DoS attack:** Makes the system unresponsive
- **Firmware tampering:** Uploads malicious code

Security Improvements

- Use **strong Wi-Fi security (WPA2/WPA3 + strong password)**
- Enable **secure communication (HTTPS/TLS)**
- Implement **authentication (API keys, login systems)**
- **Disable unused ports/services**
- Use **network segmentation (separate IoT network)**
- Apply **input validation and error handling**

Conclusion

Basic security measures like **encryption, authentication, and proper configuration** can effectively protect IoT systems from common attacks.

Q10) Your ESP32 must simultaneously interface with a DHT11 sensor, an ultrasonic sensor, an OLED display, a relay module, and a soil moisture sensor. What hardware or software conflicts could occur, and how would you solve them?

Ans- **Hardware Conflicts**

- **GPIO conflicts:** Limited or restricted pins
→ *Solution:* Plan pin mapping carefully
- **Voltage mismatch (3.3V vs 5V):**
→ *Solution:* Use level shifters or voltage dividers
- **Power issues:** High current from relays/sensors
→ *Solution:* Use external regulated supply
- **Noise/interference:** Affects analog sensors
→ *Solution:* Use filtering and proper grounding

Software Conflicts

- **Blocking code (timing issues):**
→ *Solution:* Use non-blocking code (`millis()`) or multitasking
- **I2C resource sharing (OLED):**
→ *Solution:* Use unique addresses or I2C multiplexer
- **Library conflicts:**
→ *Solution:* Use compatible libraries
- **ADC noise:**
→ *Solution:* Calibrate and use averaging/filtering

Conclusion

Conflicts arise due to **limited resources and timing issues**, and can be managed through **proper hardware planning and efficient software design**.

Q11) An ESP32 uploads sensor data to the cloud every minute. The internet connection fails for 12 hours. Design a strategy that ensures no important data is permanently lost.

Ans- **Strategy: Store and Forward**

To avoid data loss during network failure, I would use a **store locally, send later** approach.

1. Local Storage

- Store data in **SPIFFS/LittleFS or SD card**
- Each entry includes **timestamp + sensor values**
- Ensures data is safe even during power loss

2. Data Buffering

- Maintain a **FIFO queue/log file**

- Store data in compact format (CSV/JSON)

3. Network Monitoring

- If Wi-Fi fails → **store data (offline mode)**
- If restored → **switch to upload mode**

4. Data Upload (Recovery)

- Upload stored data in **batches**
- Delete data only after **successful upload**

5. Reliability Measures

- Use **timestamps + acknowledgments**
- Implement **retry mechanism**

Conclusion

This method ensures **no data loss** by combining **local storage, buffering, and reliable retransmission** after connectivity is restored.

Q12) Your IoT device must operate continuously for five years without human intervention. What changes would you make to both the hardware and software to improve long-term reliability?

Ans- For a device that must run continuously for five years without human intervention, I would focus on making it robust, low-maintenance and self-recovering.

Hardware Improvements

- Use industrial-grade components for longer life
- Ensure a stable power supply with surge protection and backup
- Protect the system using proper enclosures (dust, moisture, heat)
- Avoid moving parts and add redundancy for critical sensors

Software Improvements

- Implement a watchdog timer to auto-restart on failure
- Add fault detection and self-recovery mechanisms
- Ensure efficient memory usage and stable code
- Enable OTA updates for remote maintenance

Reliability Measures

- Use auto-reconnect and retry mechanisms for communication

- Store data locally during failures
- Perform long-term testing before deployment

Conclusion

In short, the system should be designed to be durable, fault-tolerant, and capable of handling failures on its own, ensuring reliable operation over long periods.

Q13) Can an IoT system be considered "smart" if it does not use Artificial Intelligence? Justify your answer with suitable examples. What differentiates "smart" from "intelligent"?

Ans- **Can IoT be "Smart" without AI?**

Yes. An IoT system is considered *smart* if it can **sense, process, and respond automatically** using predefined rules.

Examples:

- Irrigation system → turns ON pump at low moisture
- Smart lights → turn OFF when no motion
- Thermostat → maintains fixed temperature

These use **rule-based logic**, so they are smart but not intelligent.

Smart vs Intelligent

Smart System:

- Works on **fixed rules (if-then logic)**
- No learning or adaptation

Intelligent System:

- Uses **AI/ML**
- **Learns and adapts** over time
- Can make predictions

Conclusion

A system can be **smart without AI**, but it becomes **intelligent only when it can learn and adapt**.

Q14) Compare polling and interrupt-based programming in embedded systems. In which situations would interrupts significantly improve the performance of an IoT device?

Ans- **Polling vs Interrupts in Embedded Systems**

Polling:

- CPU continuously checks device status
- Simple but **wastes processing time**
- **Slower response** to sudden events
- *Example:* Repeatedly checking a button in a loop

Interrupt-Based Programming:

- CPU runs normal tasks and reacts only when triggered
- **Efficient and fast response**
- Supports **better multitasking**
- Slightly more complex
- *Example:* Button press triggers an interrupt function

When Interrupts Improve IoT Performance

- **Real-time response needed** (e.g., motion detection)
- **Unpredictable events** (e.g., sensor signals)
- **Low power operation** (wake on interrupt)
- **Multiple tasks running** simultaneously
- **Precise timing required**

Q15) Every engineering design involves trade-offs. Looking at your own IoT project, identify three design decisions where you sacrificed one aspect (such as cost, speed, power, accuracy, or complexity) to improve another. Explain whether you believe those trade-offs were justified.

Ans- **Key Trade-offs in My IoT Project**

1. Cost vs Accuracy

- Used **DHT11/DHT22** instead of high-precision sensors
- Lower cost, slightly less accurate
- *Justified* for basic monitoring needs

2. Simplicity vs Performance

- Used **sequential (loop-based) code** instead of multitasking
- Easier to develop and debug, but less scalable
- *Justified* since system is not time-critical

3. Power vs Functionality

- Used **continuous Wi-Fi** for real-time data

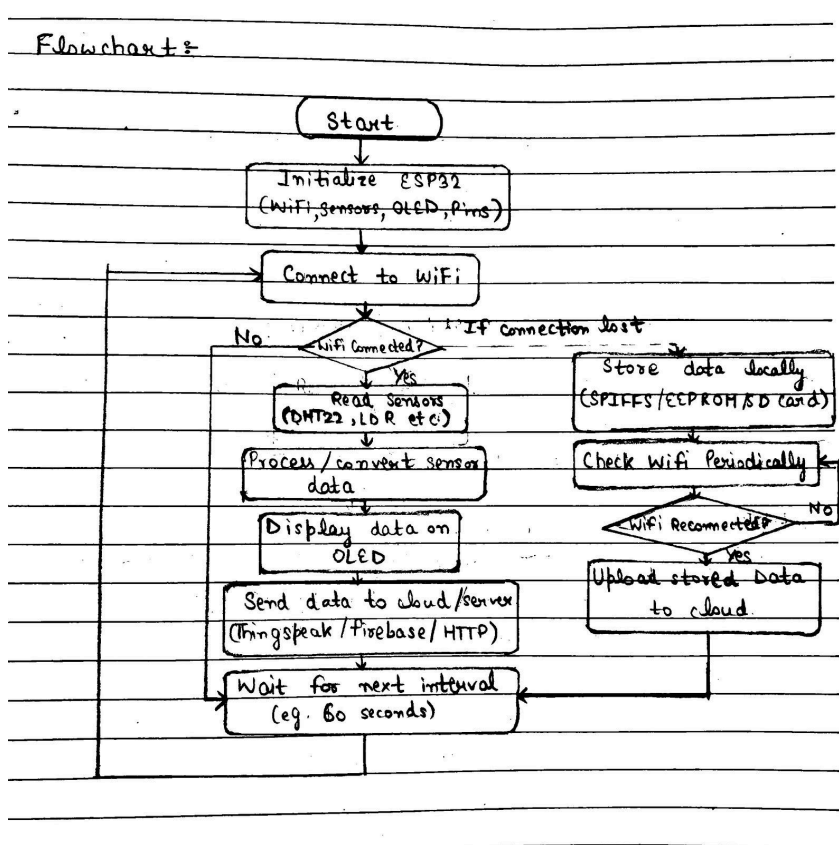
- Higher power consumption
- *Justified* for live monitoring

Conclusion

These trade-offs helped achieve a **simple, cost-effective, and functional IoT system**, which is often more practical than perfect optimization.

Q16) Create a flowchart for an IoT system that stores sensor readings locally whenever Wi-Fi is unavailable and automatically uploads the stored data once the connection is restored. Mention all the programs/protocols/I-O used in the system.

Ans-



Programs / Protocols / I-O Used

1. Hardware (I/O):

- Sensors (e.g., DHT11/DHT22) → Input
- ESP32 → Processing unit
- Flash (SPIFFS) / SD Card → Local storage
- Wi-Fi module (built-in ESP32) → Communication

2. Software Components:

- Arduino IDE / ESP-IDF code
- File system handling (SPIFFS / LittleFS)
- Wi-Fi management functions (`WiFi.begin()`, reconnect logic)
- Timer logic (`millis()` for periodic execution)

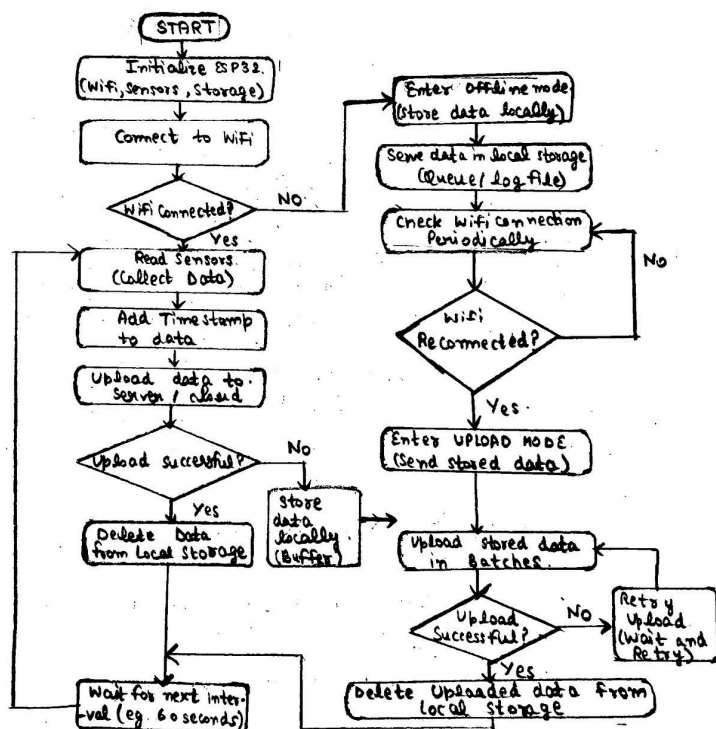
3. Communication Protocols:

- Wi-Fi (802.11) → Network connection
- HTTP / HTTPS or MQTT → Data upload to cloud
- TCP/IP → Underlying communication

Q17) Draw a flowchart showing how an ESP32 performs an OTA (Over-the-Air) firmware update, including error handling if the update fails.

Ans-

Ans - Flowchart -



Error Handling Included

- **Wi-Fi Failure:** Retry connection before checking updates
- **Download Failure:** Retry download or abort safely
- **Checksum Failure:** Reject corrupted firmware
- **Installation Failure:** Roll back to previous firmware (if supported)
- **Power Failure:** Use dual-partition OTA to prevent bricking

Conclusion

The OTA process ensures that firmware updates happen **remotely and safely**, with proper **verification and rollback mechanisms** to prevent system failure if something goes wrong.